

ServiceAPI Blocker Report

Deployment Aksha stack on 192.168.1.156 (aksha-net)

Component API_SERVICE — image dockerhubalgo/aksha_service:11022025-1 , port 4000 **Date** 2026-09-16

For the owner of the aksha_service image / ServiceAPI/ source, whoever maintains /home/algo6/Desktop/Algo6-Aksha/docker-compose.yml , and the chatbot team

Two things were found while preparing the next chatbot agents, both in the ServiceAPI (the Python FastAPI service on port 4000). The first is a deployment misconfiguration that leaves the service without a database connection and blocks two planned chatbot agents. The second is a security exposure that has nothing to do with the chatbot and is the more urgent of the two. Every statement below carries an evidence label; nothing destructive was invoked and nothing was modified.

FLAG 1 · BLOCKER

The ServiceAPI is running with no configuration at all

Its compose block sets no environment, and the image ships only the compiled binary — no prod.json , no .env . The code reads its Mongo connection string from mongodb_conn_string_prod , which nobody sets, so every database-backed route fails after a 30 s timeout on host none .

Blocks: Alert Summary and Image & Anomaly Analysis (chatbot, Phase 2); the frontend's Investigation image chat and Alert Report summary.

FLAG 2 · EXPOSURE

Port 4000 exposes root-level host operations with no authentication

Published on every interface, no auth, any-origin CORS with credentials, and the host Docker socket mounted read-write. Routes run docker system prune -f , restart every camera, return plaintext RTSP credentials, and build a docker run ... shell command from request fields.

Independent of the chatbot. Two compose-file edits reduce most of it today; the rest needs a hardened image build.

Do this first (today, ~10 minutes, one file): in docker-compose.yml , under service_api , (1) add environment: mongodb_conn_string_prod: "<the same Mongo URI node_backend uses in DATABASE>" , and (2) change ports: - "4000:4000" to - "127.0.0.1:4000:4000" . Then docker compose up -d service_api . The first unblocks the database; the second takes the service off the LAN without breaking anything that works today (details and verification in §2 and §4).

Evidence labels

Observed

executed a safe command or request and saw the result

Code

read from the deployed binary's own string constants / bytecode, or mounted source

Documented

stated in

docs/CHATBOT_SYSTEM_ARCHITECTURE.md

Inferred

reasoned from evidence; not independently reproduced

An independent adversarial-verification pass was planned for every claim here; it did not run (the multi-agent workflow hit an account spend limit — 11 of 12 agents failed to start). The labels above are the substitute: treat Inferred rows as the ones to double-check.

1. What is actually running

Compose definition	<code>/home/algo6/Desktop/Algo6-Aksha/docker-compose.yml</code> , service <code>service_api</code> , container_name: <code>API_SERVICE</code> . Mounts <code>./Aksha</code> → <code>/Aksha_Pipeline/Aksha</code> and <code>/var/run/docker.sock</code> (rw). Publishes <code>"4000:4000"</code> . No environment, no env_file: . <code>Observed</code>
Runtime environment	<code>docker inspect API_SERVICE</code> → <code>Config.Env</code> is exactly <code>PATH</code> and <code>TZ</code> . Process runs as root. <code>Observed</code>
Image contents	<code>/Aksha_Pipeline/main</code> only — a 129 MB PyInstaller one-file binary (Python 3.8, built 2026-02-11). No <code>prod.json</code> , no <code>dev.json</code> , no <code>.env</code> . The commented <code>#build: ./ServiceAPI</code> in the compose file names the source directory in the <code>Aksha-Version-2</code> repository. <code>Observed</code>
How the code configures itself	Entry script constants: <code>getenv</code> · <code>mongodb_conn_string_prod</code> · <code>pymongo.MongoClient</code> · <code>directConnection</code> · <code>connect_to_db</code> · "Database connected successfully" / "Database connection failed: ". The two GenAI modules (<code>alert_report_analyzer</code> , <code>image_analysis</code>) call <code>load_dotenv(".env")</code> and read <code>AZURE_OPENAI_API_KEY</code> , <code>AZURE_OPENAI_ENDPOINT</code> , <code>AZURE_API_VERSION</code> , <code>AZURE_DEPLOYMENT_GPT_4_TURBO</code> . <code>Code</code>
Routes (from <code>/openapi.json</code>)	<code>POST /Subscription</code> , <code>POST /Insight</code> , <code>POST /Notifications</code> , <code>POST /DockerClean</code> , <code>GET /rtsplinks.json</code> , <code>POST /Surveillance</code> , <code>POST /AlertReportAnalyzer</code> , <code>POST /ImageAnalysis</code> , <code>POST /DownloadImageAnalysisChat</code> ; plus <code>/docs</code> . No <code>securitySchemes</code> . <code>Observed</code>
Who calls it	node_backend , server-side over <code>aksha-net</code> by alias: its bundled source contains <code>http://API_SERVICE:4000/DockerClean</code> , <code>/Insight</code> , <code>/Notifications</code> , <code>/Surveillance</code> ; the access log shows <code>POST /DockerClean 200</code> nightly at 17:00Z (10–15 Sep) and <code>POST /Insight 200</code> bursts from the same container IP. <code>Code</code> <code>Observed</code> The frontend build on <code>:8080</code> calls <code>http://localhost:4000/Surveillance</code> and <code>/Insight</code> from the browser — which only works when the browser is on the server itself. <code>Observed</code>

Attribution correction: an earlier pass attributed the nightly `/DockerClean` to `cam1-Gate-Camera` because that container currently holds the source IP seen in last week's log. All containers were restarted at 04:45Z today and IPs were reassigned; the camera images contain no reference to port 4000, while the Node backend's binary does. The caller is `node_backend`.

2. Flag 1 — no configuration: root cause, reproduction, fix

Root cause chain

- The compose block gives the service no environment and no env file. `Observed`
- The image carries no config files, so `load_dotenv(".env")` finds nothing and every `getenv` returns `None`. `Observed` `Code`
- The entry script builds `MongoClient` from `getenv("mongodb_conn_string_prod")`. With the variable unset, the client resolves a host literally named `none`; `pymongo` retries for its 30 s server-selection timeout and raises `none:27017: [Errno -3] Temporary failure in name resolution`. `Code` `Observed` (the exact `None` → `none` step is `Inferred` from the error text)
- `/AlertReportAnalyzer` and `/ImageAnalysis` begin by reading a `Resource` document from Mongo to check `isGenAIFeatures` — that is why they need the database at all. Only after that do they call the Azure OpenAI modules, which need the four `AZURE_*` variables from `.env`. `Code`
- `/Insight` touches neither the environment nor the database, which is why it works today. `Code` `Observed`

Reproduction (non-destructive, safe to rerun)

```
curl -s -m 40 -X POST http://localhost:4000/AlertReportAnalyzer \
  -H 'Content-Type: application/json' \
  -d '{"filtered_data":{"cam3":{"total_alerts_generated":1}}, "start_date":"2026-09-16",
    "end_date":"2026-09-16", "model_option":"GPT 4o", "lang_option":"eng"}'
# → HTTP 500 after ~30.3 s:
# {"report_analysis": "", "message": "failed: error in the call_alert_report_analyzer function:
#   none:27017: [Errno -3] Temporary failure in name resolution, Timeout: 30s, ..."}

```

`model_option "GPT 4o"` and `lang_option "eng"` are the values the frontend itself sends (`insights/alertReport/index.jsx`, `ChatPopover.tsx`). `Code`

Fix — three parts, in order

- Database connection (unblocks the 30 s failure).** Add to the `service_api` block the variable the code reads, using the same URI `node_backend` already uses for its `DATABASE` variable

(the pipeline's `prod.json` carries the identical string as `DATABASE_CONNECTION`):

```
service_api:
  container_name: API_SERVICE
  image: dockerhubalgo/aksha_service:11022025-1
  environment:
    mongodb_conn_string_prod: "mongodb://<user>:
<password>@mongodb:27017/Aksha?authSource=admin"
  ports:
    - "127.0.0.1:4000:4000"          # see §4 — remove LAN exposure
in the same edit
  volumes:
    - ./Aksha:/Aksha_Pipeline/Aksha
    - /var/run/docker.sock:/var/run/docker.sock
  networks: [aksha-net]
  restart: always
```

Then `docker compose up -d service_api` (a few seconds of downtime; the backend's nightly call is unaffected).

- 2. **Azure OpenAI credentials (needed for the two GenAI routes to produce output).** Provide a `.env` at `/Aksha_Pipeline/.env` inside the container — e.g. a file next to the compose mounted read-only as `- ./service_api.env:/Aksha_Pipeline/.env:ro` — containing `AZURE_OPENAI_API_KEY`, `AZURE_OPENAI_ENDPOINT`, `AZURE_API_VERSION`, `AZURE_DEPLOYMENT_GPT_4_TURBO`. Keep it out of the repository.
- 3. **Feature flag.** The routes answer `failed: Gen AI Features are not enabled` unless the Mongo `Aksha.Resource` document has `isGenAIFeatures` set true. This is a product/licensing decision, not a bug.

Verification after each part

- After part 1: the reproduction above returns within ~1-2 s — either a real `report_analysis` or the fast `Gen AI Features are not enabled` message. Either proves the database is connected; the 30 s timeout is gone.
- After parts 2-3: the same call returns HTTP 200 with a non-empty `report_analysis`; `POST /ImageAnalysis` with a base64 image returns an `answer`.
- The service's own file logger appears to write nowhere visible (no `insight.log` exists in the container or the shared `Aksha` dir), so rely on the behavioural check above rather than a log line.

Observed

Owner

Image `dockerhubalgo/aksha_service:11022025-1`, built 2026-02-11 from `ServiceAPI/` in the `Aksha-Version-2` repository; deployed by `/home/algo6/Desktop/Algo6-Aksha/docker-compose.yml`. Parts 1-2 are pure deployment changes; a better long-term fix (the binary failing loudly at startup when `mongodb_conn_string_prod` is unset, and reading the Azure settings from environment rather than a file) belongs to the image owner.

3. Who needs this — endpoints, agents, and frontend features

SERVICEAPI ENDPOINT	CHATBOT AGENT (STATUS)	FRONTEND FEATURE	WORKS TODAY?
<code>POST /AlertReportAnalyzer</code>	Alert Summary (planned, Phase 2). Natural data source for a KPI Report agent too.	Insights → Alert Report summary (<code>insights/alertReport/index.jsx</code>). Dead in the :8080 build for a second reason: <code>REACT_APP_AlertReportAnalyzer</code> is missing from <code>.env.prod</code> , so the bundle posts to the literal string <code>undefined</code> .	No — 500 after 30 s (Flag 1)
<code>POST /ImageAnalysis</code> <code>POST /DownloadImageAnalysisChat</code>	Image & Anomaly Analysis (planned, Phase 2).	Investigation → image chat and chat-log download (<code>investigation/ChatPopover/ChatPopover.tsx</code>). Dead in the :8080 build for the same <code>.env.prod</code> reason.	No — same failure

SERVICEAPI ENDPOINT	CHATBOT AGENT (STATUS)	FRONTEND FEATURE	WORKS TODAY?
POST /Insight	None built. A possible enhancement for Insights & Analytics / KPI Report.	Heatmap generation: server-side via node_backend 's /api/insight ; also browser-direct as REACT_APP_INSIGHT=http://localhost:4000/Insight (works only from a browser on the server).	Yes
POST /Surveillance , POST /Notifications , POST /DockerClean , POST /Subscription , GET /rtsplinks.json	None, by design — write/control/credential operations are Phase 3, approval-gated (service_api_client.py 's docstring already rules them out).	Camera start/restart/update (browser-direct localhost:4000/Surveillance) and node_backend 's nightly prune / camera restarts (server-side).	Yes

Built chatbot agents affected: none. All nine live agents (camera_operations, alert_investigation, live_monitoring, insights_analytics, notification, help_guide, error_explanation, camera_troubleshooting, timeline) read only the Node API on :5000 or local documentation; service_api_client.py exists but is not imported anywhere. Code

4. Flag 2 — exposure on port 4000

Exposure facts

- Published on every interface:** 0.0.0.0:4000 and [::]:4000 (compose "4000:4000"). The host has a LAN address (192.168.1.156/24 — the one the production frontend points browsers at) and a Tailscale address (100.72.172.28). Observed
- No authentication:** no securitySchemes in the schema; an empty-body POST /ImageAnalysis reaches Pydantic validation (HTTP 422), so no auth layer runs before handlers. Observed
- Permissive CORS:** a preflight with Origin: http://attacker.example is answered with that origin reflected, access-control-allow-credentials: true , all methods — the bodiless routes are CORS "simple requests" and can be fired from any web page a person on the host or LAN visits. Observed
- Root + host Docker socket:** /var/run/docker.sock is mounted read-write, the process is uid 0, and the image ships the docker CLI. Whoever controls this process controls the host's Docker daemon. Observed
- Host firewall does not help by default:** ufw is active with default-DROP, but Docker's published ports bypass ufw's INPUT chain unless DOCKER-USER rules exist — which could not be read without sudo. Observed Inferred
- Swagger UI and the full schema** are served to anyone (/docs , /openapi.json). Observed

Findings

ROUTE	SEVERITY	WHAT IT DOES	BASIS
POST /Surveillance body: type , camera_list[]	Critical	Builds a shell command from request fields — docker run -d --restart unless-stopped --env ENVIRONMENT="prod.json" --env CAMERA_NAME=... --env RTSP_LINK="..." --env MY_ALERTS="..." ... --volumes-from API_SERVICE — and runs it through os.system as root with the host socket; type selects start / restart (docker restart) / stop (docker rm -f). Unauthenticated string interpolation of user input into a root shell command is a command-injection surface; the exact quoting is not visible in constants, so treat as critical until the source is reviewed. Stopping every camera is a persistent surveillance blackout. Not exploited or tested here.	Code
POST /DockerClean no body	High	Runs docker system prune -f on the host daemon: deletes every stopped container (a camera stopped through the UI is gone), dangling images, unused networks and build cache. Called nightly at 17:00Z by node_backend — legitimate housekeeping that must be preserved if the route is removed.	Code Observed

ROUTE	SEVERITY	WHAT IT DOES	BASIS
POST /Notifications no body	High	docker restart of every running camera ("Running cameras are restarted"): a detection blackout for the restart + model-load window, repeatable in a loop, triggerable as a CSRF simple request.	Code
GET /rtsplinks.json	High	Returns Aksha/rtsplinks.json — the file §5.6 of the architecture doc documents as plaintext RTSP camera credentials (607 bytes, present on the host). The same file is also served unauthenticated by the Node API on :5000 (GET /rtsplinks.json → 200, 607 bytes), which the frontend depends on — locking down :4000 alone does not close this.	Documented Observed
Container config: docker.sock rw, root	High	The amplifier behind every row above; any code-execution bug in the compiled binary is a full host compromise. The same socket is inherited by every camera container via --volumes-from API_SERVICE , and mounted into anomaly_trainer .	Observed Code
CORS: reflect any origin + credentials	Medium	Turns network exposure into drive-by exposure: a malicious page can make a user's browser fire the bodiless POSTs at localhost:4000 / 192.168.1.156:4000 .	Observed
POST /Subscription	Medium	Unauthenticated write of the product licence key (subscriptionKey ; constants: Running / Status / RemainingDays / Expired). Could invalidate the installation's subscription state until an operator re-enters the key.	Code
POST /AlertReportAnalyzer , POST /ImageAnalysis	Low → Medium once fixed	Broken today (Flag 1): each request only ties up a worker for 30 s. Once repaired they become an unauthenticated, network-reachable proxy to a paid vision/LLM deployment — re-rate when Flag 1 is closed.	Observed
POST /Insight , POST /DownloadImageAnalysisChat	Low	Unauthenticated per-camera activity data and on-demand CPU/disk use; no credentials involved.	Observed
/docs , /openapi.json	Info	A click-to-execute UI for every route above. Harmless alone; lowers the skill needed to abuse the rest.	Observed
Node API :5000 — GET /api/SurveillanceStatus	Needs review	Answered 200 {"success":true,"message":"success to perform Surveillance operation"} to an unauthenticated GET with no parameters. No corresponding ServiceAPI call appeared in the log, so it may be a no-op — but the Node backend is a second front door to the same operations (it calls API_SERVICE:4000/Surveillance and /Notifications) and its route protection should be confirmed in the repository audit.	Observed

Remediation, in order

#	WHEN	ACTION	CLOSES	WHAT IT WOULD BREAK
1	Immediate	Bind the port to loopback: ports: - "127.0.0.1:4000:4000" in the service_api block; docker compose up -d service_api . Verify with docker port API_SERVICE (expect 127.0.0.1:4000 only) and, from another LAN device, curl -m 5 http://192.168.1.156:4000/openapi.json (expect refused/timeout).	LAN and Tailscale reachability of every route; LAN-origin CSRF.	Nothing that works today. node_backend reaches the service by alias over aksha-net ; the frontend already hard-codes localhost:4000 , so only a browser on the server itself ever worked — and still will. Anyone using <tailscale-ip>:4000 for admin loses that path (see #4).
2	Immediate	Host-level backstop Docker cannot bypass: first record the current state with sudo iptables -S DOCKER-USER and sudo ufw status verbose , then sudo iptables -I DOCKER-USER -i enp2s0 -p tcp --dport 4000 -j DROP (and -i tailscale0); consider the same for 27017 and 5000. Persist via ufw's after.rules DOCKER-USER block or iptables-persistent .	Reachability, independent of future compose edits.	Needs root; rules must go in DOCKER-USER , not ufw's INPUT chain, or they do nothing for Docker ports.

#	WHEN	ACTION	CLOSES	WHAT IT WOULD BREAK
3	Immediate	Treat the RTSP credentials as exposed: rotate each camera's password, update <code>Aksha/rtsplinks.json</code> and each camera's stored <code>RTSP_LINK</code> (recreate through the app's own Surveillance update/restart flow, one camera at a time), set the file to <code>0600</code> , and gate or remove the Node API's unauthenticated <code>GET /rtsplinks.json</code> on <code>:5000</code> .	<code>/rtsplinks.json</code> on both ports; architecture-doc §5.6 gap.	Each camera drops out during its recreate. Locking the <code>:5000</code> route breaks the camera-directory and alert-modal readers unless the plain-axios calls (<code>cameraDirectory/List/index.jsx</code> , <code>cameraGroup/List/List.jsx</code>) switch to the JWT-carrying <code>axiosJWT</code> first.
4	Soon	Proxy only the routes the UI needs through the frontend nginx (<code>AkshaV2-UIUX/frontend/nginx/conf.d/default.conf</code>): <code>/svc/Surveillance</code> and <code>/svc/Insight</code> (and the three analysis routes only when revived), each with an allow-list and an <code>auth_request</code> against the Node API. Deliberately add no location for <code>/DockerClean</code> , <code>/Notifications</code> , <code>/rtsplinks.json</code> , <code>/Subscription</code> , <code>/docs</code> . Point <code>.env.prod</code> at the relative paths and rebuild.	Makes camera start/stop and heatmaps work for users not sitting at the server (they silently fail today); moves <code>/Surveillance</code> from "anyone on the network" to authenticated UI users; removes the need for permissive CORS.	A frontend rebuild and a small nginx change. Without the auth step you have only moved "anyone can stop all cameras" to <code>:8080</code> — still a large improvement, not the end state.
5	Soon	Hardened ServiceAPI build (image owner): auth dependency on every route (shared secret header, or verify the Node JWT); remove <code>/DockerClean</code> and <code>/Notifications</code> from HTTP or restrict to source <code>172.18.0.0/16</code> + a secret <code>node_backend</code> also receives; build the <code>docker run</code> command with the Docker SDK / argument lists, never <code>os.system</code> string templating; CORS with an explicit origin list and no credentials; <code>docs_url=None</code> in production; exit loudly at startup when <code>mongodb_conn_string_prod</code> is unset; read Azure settings from environment. Move the nightly prune to a host cron and drop the backend's call.	Every route-level finding, including the root cause of Flag 1.	A coordinated release of <code>aksha_service</code> and <code>node_backend</code> (the backend's four hard-coded ServiceAPI calls). Until it lands, #1–#4 are the compensating controls.
6	Later	Shrink the socket blast radius: a docker-socket-proxy sidecar on <code>aksha-net</code> allowing only container start/stop/restart/create and image pulls (no <code>system</code> , <code>volumes</code> , <code>exec</code>), with the ServiceAPI pointed at it via <code>DOCKER_HOST</code> ; run the service as a non-root user.	Worst case from any bug behind <code>/Surveillance</code> ; makes <code>/DockerClean</code> a no-op by construction.	The compiled binary must honour <code>DOCKER_HOST</code> (docker CLI and SDK both do; unconfirmed for this build). Each Surveillance action type needs a test pass against the proxy's allow-list.

5. Method and limits

- **Read-only.** Commands used: `docker inspect`, `docker logs -t`, `docker port`, `docker exec ... ls/env/grep`, `docker network inspect`; `GET /openapi.json`; one valid `POST /AlertReportAnalyzer` and one valid `POST /Insight` (analysis routes); one empty-body `POST /ImageAnalysis` (validation only). **Never sent:** any request to `/DockerClean`, `/Notifications`, `/Surveillance` (beyond one empty-body 422 that cannot execute a handler), `/Subscription`, or `/rtsplinks.json`. No file, container, image or network was changed.
- **Binary analysis.** The ServiceAPI's PyInstaller archive was parsed with the standard library only (table of contents + `zlib`) to read the entry script's string constants; the extracted module bytecode under `/tmp/_MEI...` was string-scanned; the Node backend's `pkg` bundle was string-searched. Nothing was executed or unmarshalled beyond the archive index.
- **Redaction.** Every credential encountered (Mongo URIs, cloud keys, OAuth secrets) was redacted at the point of reading and appears nowhere in this report.
- **Limits.** The compiled binaries hide exact quoting and control flow (hence the `Inferred` labels on the injection potential and the `None` → `none` step); firewall rules were unreadable without `sudo`; the planned adversarial verification did not run. The repository audit (pending access) can settle each of these against source.

Prepared 2026-09-16 during the Aksha chatbot Phase 2 work (see [docs/CHATBOT_SYSTEM_ARCHITECTURE.md](#) §9k). A full [Aksha-Version-2](#) repository audit — version correspondence of the deployed images, backend/frontend contract compatibility with the chatbot, and source review of the items marked "needs review" above — is a separate report, pending repository access.